

IoT Monitoring System

July 6, 2026

Contents

1	Iterazione 0	3
1.1	Diagramma dei Casi d'Uso	3
1.2	Casi d'Uso	3
1.3	Requisiti Funzionali	5
1.4	API	5
1.5	Deployment/Topology Diagram	6
1.6	Toolchain di sviluppo	7
1.6.1	Toolchain embedded	7
1.6.2	Toolchain backend	7
1.6.3	Toolchain dashboard	7
1.6.4	Toolchain di test, debug e supporto	7
2	Iterazione 1	9
2.1	Setup dell'ambiente di sviluppo e toolchain	9
2.2	Architettura software	9
2.3	Deployment/Topology Diagram	11
2.4	Component Diagram	13
2.5	Domain Class Diagram	15
2.6	Interface Diagram	16
3	Piano di Qualità del Progetto	18
3.1	Obiettivi di qualità	18
3.2	Strategia generale di verifica e validazione	18
3.2.1	Analisi statica	18
3.2.2	Analisi dinamica	18
3.2.3	Test funzionali e di integrazione	19
3.3	Qualità del backend Java	19
3.3.1	Analisi statica del backend	19
3.3.2	Test automatici del backend	20
3.3.3	Test funzionali del backend	20
3.4	Qualità della dashboard web	20
3.4.1	Strategia di verifica della dashboard	20
3.5	Qualità del firmware embedded	21
3.5.1	Strategia di verifica del firmware	21
3.6	Piano dei test per componente	22
3.6.1	Backend Java	22
3.6.2	Dashboard	22
3.6.3	Firmware embedded	23
3.7	Tracciabilità tra requisiti e test	23
3.8	Criteri di completamento del piano di qualità	23
3.9	Motivazione della strategia adottata	23

4	Iterazione 1 – Draft dell’architettura software e setup dell’ambiente di sviluppo	25
4.1	Obiettivi dell’iterazione	25
4.2	Setup dell’ambiente di sviluppo e toolchain	25
4.3	Scelta architetturale	26
4.4	Architectural Rationale	26
4.5	Introduzione delle interfacce	28
4.6	Vista di deployment / topology	29
4.7	Vista a componenti	29
4.8	Vista statica a classi	29
4.9	Incremento software realizzato	29
4.10	API REST del sistema	30
4.10.1	API device-server	32
4.10.2	API dashboard-server	32
4.11	Legenda dei diagrammi architetturali	32
4.12	Corrispondenza tra diagrammi e implementazione	33
4.13	Considerazioni finali sull’iterazione	34
5	Iterazione 1 - Domain Class Diagram e Interface Diagram	35
6	Introduzione della DASHBOARD - Architettura software del backend	36
6.1	Gestione della telemetria	36
6.2	Gestione dei dispositivi	36
6.3	Gestione delle soglie e generazione allarmi	37
6.4	Gestione dei comandi remoti	37
7	Introduzione della view e dashboard web	37
7.1	Motivazioni progettuali	38
8	ITERAZIONE 1.1 - MODIFICA DEI DIAGRAMMI PER L’INTRODUZIONE DELLA DASHBOARD	38
8.1	Aggiornnamento del Component Diagram	38
8.2	Architettura complessiva del sistema	38
8.3	Scelta della layered architecture	39
8.4	Introduzione della componente View	39
8.5	Uso delle interfacce nei layer applicativi	39
8.6	Sintesi della struttura progettuale	40

1 Iterazione 0

1.1 Diagramma dei Casi d'Uso

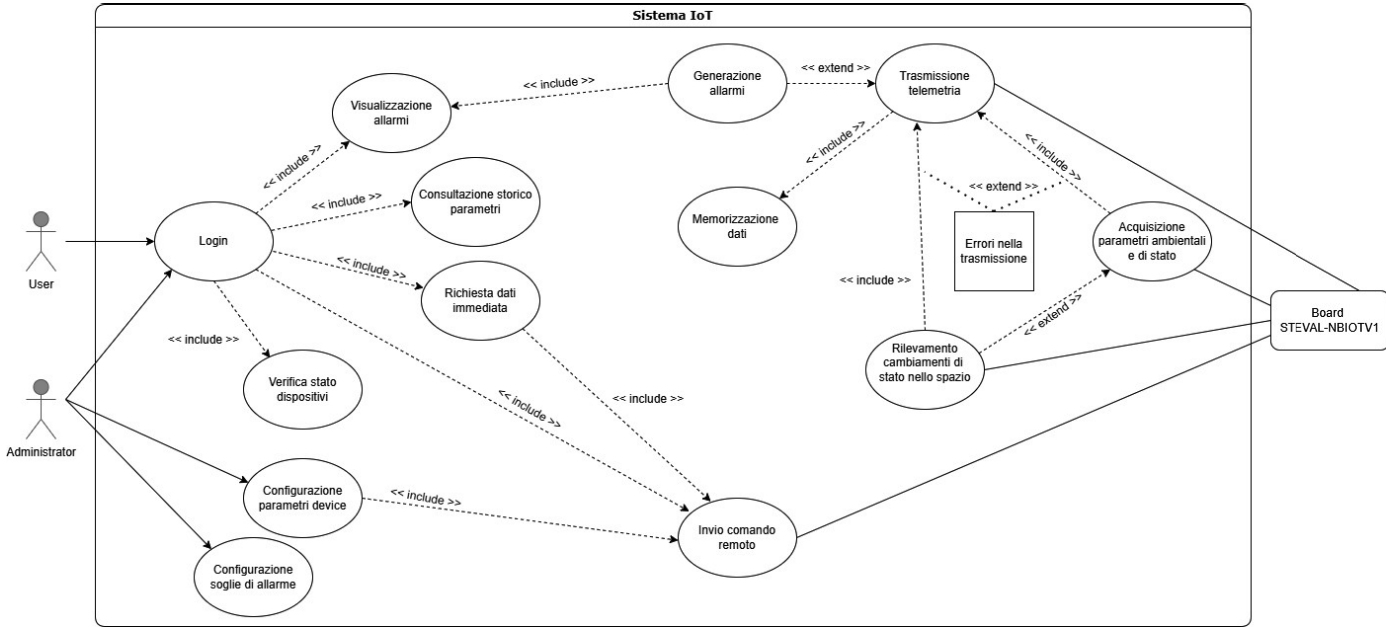


Figure 1: IoT System-v5

1.2 Casi d'Uso

- **UC1: Acquisizione parametri ambientali e di stato** → La board (device/sensore) acquisisce periodicamente i valori dei sensori integrati (Temperatura, Umidità, Pressione) e i dati di stato del dispositivo (es. orientamento nello spazio, % batteria, se è online/offline). [Device]
- **UC2: Trasmissione telemetria** → La board invia i dati acquisiti al server via HTTP o MQTT. [Device]
- **UC3: Memorizzazione dati** → Il server memorizza i dati ricevuti in un database. [Server]
- **UC4: Errori nella trasmissione** → In caso di mancata ricezione del pacchetto, il device effettua la ritrasmissione dei dati. [Device]
- **UC5: Consultazione storico parametri** → L'utente è in grado di consultare lo storico delle misure effettuate dai dispositivi tramite interfaccia grafica. [Dashboard-Server]
- **UC6: Verifica stato dispositivi** → L'utente è in grado di controllare lo stato dei dispositivi connessi (online/offline, %batteria, orientamento nello spazio). [Dashboard-Server]
- **UC7: Rilevamento cambiamenti di stato nello spazio** → Il dispositivo è in grado di rilevare urti, cadute, inclinazioni e genera un evento (invia messaggio di allerta). [Device-Server]
- **UC8: Visualizzazione allarmi** → L'utente consulta gli allarmi generati dal dispositivo a seguito di un evento specifico (superamento valore soglia o caduta/impatto).
- **UC9: Configurazione parametri device** → L'amministratore imposta i parametri di funzionamento del dispositivo. [Administrator-Server/Device]
- **UC10: Invio comando remoto** → Possibilità di comandare il dispositivo (board) a distanza tramite l'invio di appositi comandi (es. riavvio, generazione di un particolare evento). [User/Administrator-Server/Device]

- *UC11*: **Richiesta dati immediata** → L'utente è in grado di richiedere una misurazione immediata dei dati col conseguente inoltro verso il server. [User-Server/Device]
- *UC12*: **Configurazione soglie di allarme** → L'amministratore imposta i valori di soglia per la generazione di un allarme. [Administrator-Server]
- *UC13*: **Generazione allarmi** → Il sistema genera un evento di allarme al superamento delle soglie impostate, il quale viene visualizzato tramite interfaccia grafica. [Server]
- *UC14*: **Login** → Accesso al sistema tramite apposite credenziali.

1.3 Requisiti Funzionali

- **RF1: Acquisizione parametri ambientali** → La board acquisisce periodicamente i valori di Temperatura, Umidità e Pressione tramite i sensori presenti su di essa.
- **RF2: Acquisizione parametri di stato** → La board acquisisce dati legati allo stato dispositivo, quali la % della batteria, l'identificativo, se è online o offline e l'inclinazione nello spazio.
- **RF3: Identificazione misurazioni** → Ad ogni misurazione effettuata, viene associata un'identificazione univoca tramite l'aggiunta di un timestamp e dell'ID del dispositivo che ha raccolto le informazioni.
- **RF4: Trasmissione telemetria** → La board trasmette i dati al server tramite comunicazione HTTP o MQTT.
- **RF5: Gestione degli errori** → In caso di mancata ricezione del pacchetto (mancata conferma), avviene la ritrasmissione.
- **RF6: Salvataggio telemetrie** → Le telemetrie ricevute dal dispositivo vengono salvate in un database persistente (SQLite).
- **RF7: Consultazione parametri** → Il sistema permette la consultazione di tutti i dati salvati nel database tramite interfaccia grafica.
- **RF8: Generazione allarmi** → Il server invia messaggi d'allarme nel caso in cui vengono superati i valori soglia impostati.
- **RF9: Visualizzazione allarmi** → La dashboard permette la visualizzazione dei parametri con valore fuori range.
- **RF10: Configurazione delle soglie** → L'amministratore è in grado di impostare i valori dei parametri superati i quali verrà generato un messaggio di allarme.
- **RF11: Lettura immediata** → Il sistema permette un'interazione tramite dashboard o tramite pulsante fisico sulla board per conoscere immediatamente il valore dei dati ambientali.
- **RF12: Invio dei comandi remoti** → Controllo della board tramite comandi eseguiti tramite interfaccia grafica.
- **RF13: Servizio dashboard** → il backend hosta una dashboard che sfrutta le API REST per visualizzare e gestire il sistema.
- **RF14: Registrazione nuovi dispositivi** → Il sistema supporta la registrazione di più dispositivi attribuendo ad ognuno un identificativo univoco.
- **RF15: Rilevamento eventi movimento/urto** → La board trasmette un messaggio di avviso qualora avviene uno spostamento nello spazio o eventi anomali come caduta o urto.
- **RF16: Controllo stato dispositivo** → Il sistema permette di controllare lo stato dei dispositivi collegati alla rete.
- **RF17: Accesso al sistema:** Possibilità di accesso al sistema mediante apposite credenziali (Utente/Amministratore).

1.4 API

- **API** per l'invio delle telemetrie dal dispositivo al server via HTTP o MQTT, quindi le interazioni tra board e server.
- **API** per l'interazione dell'utente col sistema tramite dashboard, quindi tutta la gestione dashboard-server.

1.5 Deployment/Topology Diagram

Il **Deployment/Topology Diagram** rappresenta la configurazione del sistema a runtime e mostra come i principali componenti software sono distribuiti sui nodi fisici del sistema. L'architettura del sistema proposto è di tipo **client-server distribuita**, articolata su tre nodi principali:

1. il dispositivo *embedded*, basato su board **STEVAL-NBIOTV1**;
2. il *nodo server*, ospitato su **Raspberry Pi**;
3. il *nodo client*, utilizzato dall'utente o dall'amministratore per l'accesso alla *web dashboard*.

La board **STEVAL-NBIOTV1** costituisce il nodo di acquisizione dati. Su di essa è eseguito un firmware sviluppato in linguaggio C, responsabile della lettura periodica dei sensori integrati, della raccolta dei parametri di stato del dispositivo e della trasmissione della telemetria verso il server. Il firmware può inoltre gestire il *polling* (ossia richieste ad intervalli regolari) di eventuali comandi remoti e la rilevazione di eventi anomali, come urti o cambiamenti di orientamento nello spazio.

Il **Raspberry Pi** rappresenta il nodo centrale del sistema. Su di esso risiedono il backend applicativo (sviluppato in Python o in Java), le API REST per la comunicazione col dispositivo e la dashboard, e il database SQLite utilizzato per la persistenza delle telemetrie, degli allarmi e delle configurazioni. Sullo stesso nodo viene inoltre eseguita una dashboard web che permette all'utente e all'amministratore di consultare lo stato del sistema, visualizzare lo storico dei dati e inviare comandi remoti ai dispositivi. In una possibile estensione architetturale, il **Raspberry Pi** (backend) potrebbe ospitare anche un broker MQTT per la comunicazione *publish/subscribe* con i device.

Infine, l'interazione dell'utente col sistema è possibile grazie ad una **dashboard web**, rappresentata dal nodo client. L'utente è in grado di inviare comandi al dispositivo e visualizzare i dati raccolti accedendo alla dashboard tramite un PC o altro dispositivo dotato di browser web.

La comunicazione tra browser web e server backend avviene tramite protocollo HTTP(S), mentre la comunicazione tra il server e la board **STEVAL-NBIOTV1** potrebbe essere implementata o anch'essa tramite HTTP(S), o via MQTT su rete IP.

Dal punto di vista topologico, come illustrato nel **Deployment/Topology Diagram** in figura 2, il **Raspberry Pi** svolge il ruolo di punto centrale di aggregazione dei dati e di coordinamento del sistema, mentre il dispositivo embedded costituisce il nodo periferico di sensing e attuazione. L'utente interagisce esclusivamente col backend senza accedere direttamente al database, questo grazie ad una scelta architetturale a più livelli che consente di separare acquisizione, elaborazione, persistenza e presentazione.

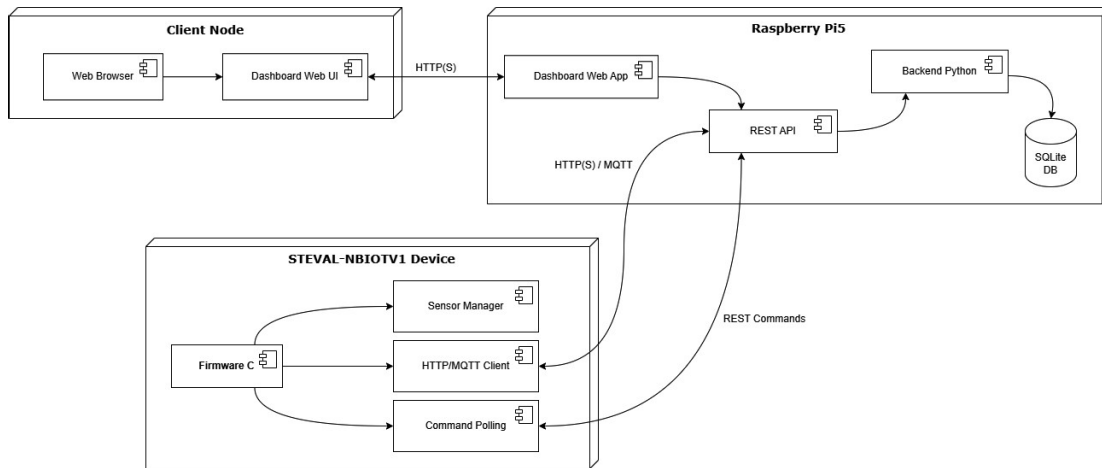


Figure 2: Deployment Diagram

1.6 Toolchain di sviluppo

La realizzazione del sistema richiede l'adozione di una toolchain (strumenti di sviluppo) eterogenea, coerente con la natura distribuita del progetto e con la presenza di componenti **embedded**, **server-side** (*backend*) e **client-side** (*frontend*).

1.6.1 Toolchain embedded

Il firmware della board **STEVAL-NBIOTV1** sarà sviluppato in linguaggio C utilizzando un ambiente di sviluppo per sistemi embedded, in particolare **STM32CubeIDE**. Le librerie software messe a disposizione dalla piattaforma consentono la gestione dei sensori integrati, delle periferiche di comunicazione e dei protocolli applicativi necessari all'invio della telemetria.

Il caricamento del firmware e le attività di debug sulla board avvengono attraverso strumenti di flashing e di debugging integrati nella piattaforma di sviluppo ST. L'output prodotto dall'esecuzione del codice potrà essere osservato attraverso uno specifico software di comunicazione seriale, come **Tera Term** o **PuTTY**.

1.6.2 Toolchain backend

La componente server sarà sviluppata utilizzando un linguaggio **Python** oppure **Java** e ospitata sul **Raspberry Pi5**. Il backend può essere realizzato mediante framework come *Flask* o *FastAPI*, nel caso di Python, oppure mediante **Spring Boot** con **Maven** nel caso di Java. Tali tecnologie consentono di esporre **API REST** per la ricezione dei dati provenienti dal dispositivo embedded e per la consultazione delle informazioni tramite dashboard.

La persistenza dei dati sarà gestita tramite **SQLite**, accessibile in Python mediante il modulo **sqlite3**, mentre in Java attraverso l'interfaccia **JDBC**. Per la gestione delle dipendenze invece, in ambiente Python si prevede l'impiego di **pip** e di un ambiente virtuale **venv**, mentre nel caso di Java si sfrutterà **Maven** aggiungendo le varie dipendenze al file **pom.xml**.

1.6.3 Toolchain dashboard

La dashboard sarà implementata come applicazione web e potrà essere sviluppata utilizzando **HTML**, **CSS** e **JavaScript**. Essa sarà servita direttamente dal backend e interagirà col sistema tramite chiamate **API REST**.

Per la visualizzazione grafica dei dati storici è possibile utilizzare librerie **JavaScript** dedicate, come **Chart.js**. In alternativa, nel caso di un backend sviluppato in Java con **Spring Boot**, sarà possibile integrare la parte di presentazione direttamente nell'applicazione server.

1.6.4 Toolchain di test, debug e supporto

Il test delle API potrà essere effettuato mediante strumenti software come **Postman** oppure direttamente da terminale tramite comando **curl**. Per quanto riguarda il debug della dashboard e delle interazioni *client-server*, potrà essere svolto mediante gli strumenti di sviluppo integrati direttamente del browser oppure tramite ambienti di supporto allo sviluppo come **CodeMR**.

Per il backend, le attività di test potranno essere svolte con **JUnit** nel caso di Java, mentre in Python il logging applicativo consentirà di visualizzare le varie telemetrie ricevute e l'esecuzione dei comandi remoti.

Per il versionamento del codice sorgente e della documentazione si utilizzerà **Git**, con repository ospitati su **GitHub**. La modellazione architetturale e la rappresentazione dei diagrammi potranno essere effettuate tramite strumenti UML o di diagrammazione, come **draw.io** o **StarUML**, mentre la redazione della documentazione tecnica potrà avvenire in $\text{\LaTeX}$, tramite piattaforme come **Overleaf** o editor locali come **TeXstudio**.

Componente	Strumenti / Tecnologie	Descrizione
Embedded	STM32CubeIDE, linguaggio C, Tera Term	Sviluppo del firmware della scheda STEVAL-NBIOTV1 (client), gestione dei sensori, delle periferiche di comunicazione e debug tramite interfaccia seriale.
Backend	Python / Java, Flask / FastAPI / Spring Boot, Maven, SQLite	Realizzazione della componente server su Raspberry Pi5, esposizione di API REST, gestione della logica applicativa e persistenza dei dati.
Dashboard	HTML, CSS, JavaScript, Chart.js	Implementazione dell'interfaccia web per la visualizzazione dei dati e interazione con il backend tramite API REST.
Test e debug	Postman, curl, strumenti di sviluppo del browser, JUnit, logging	Verifica del corretto funzionamento delle API, debug delle interazioni client-server e del comportamento applicativo.
Versionamento e documentazione	Git, GitHub, draw.io, StarUML, Overleaf, TeXstudio	Gestione del versionamento del codice, modellazione architetturale e redazione della documentazione tecnica in LaTeX.

Table 1: Toolchain di sviluppo del sistema

2 Iterazione 1

2.1 Setup dell'ambiente di sviluppo e toolchain

Per lo sviluppo del progetto è stata definita una toolchain eterogenea, coerente con la natura distribuita del sistema e con la presenza sia di una parte embedded (i device) sia di una parte server-side (backend), che ospiterà sia la logica applicativa sia l'interfaccia grafica accessibile via browser web.

- **STM32CubeIDE**: utilizzato per lo sviluppo del firmware embedded in linguaggio C sulla board **STEWAL-NBIOTV1**.
- **Eclipse IDE**: utilizzato per lo sviluppo del backend Java.
- **Spring Boot**: framework adottato per la realizzazione del backend HTTP/REST e della logica lato server.
- **Maven**: strumento utilizzato per la gestione delle dipendenze, la compilazione e il packaging dell'applicazione.
- **SQLite**: database relazionale scelto per la persistenza locale sul Raspberry Pi5.
- **DB Browser for SQLite**: strumento di supporto per l'ispezione e il debug del database.
- **Postman**: utilizzato per il test manuale delle API REST.
- **GitHub**: utilizzato per il versionamento del codice e la condivisione degli artefatti progettuali.
- **Raspberry Pi5**: nodo server di destinazione per l'implementazione finale del backend.

2.2 Architettura software

La scelta dell'architettura software si è basata sul principio di funzionamento di un sistema IoT. Esso è caratterizzato da una rete interconnessa di dispositivi dotati di specifici sensori, il cui compito è quello di acquisire informazioni relative a grandezze fisiche esterne e di trasmetterle verso un nodo centrale. Il compito di questo nodo sarà quello di ricevere ed elaborare i dati inviati dai device embedded, garantendo persistenza e svolgendo il ruolo di coordinatore tra i vari dispositivi presenti sulla rete.

Scelta di un'architettura client-server Lo scenario appena descritto corrisponde al paradigma **client-server**, il quale sarà infatti alla base dello sviluppo di questa applicazione di monitoraggio IoT. Nello specifico, i vari nodi embedded rappresentano la parte **client** dell'architettura, mentre il sistema centrale (backend) rappresenta la parte **server**.

Oltre ai vari dispositivi IoT, anche l'utente o l'amministratore che interagisce col sistema tramite dashboard web rappresenta un nodo **client** dell'architettura, il quale è in grado di comunicare col **server** attraverso apposite **API REST** esposte da quest'ultimo.

La scelta di adottare un'architettura **client-server** risulta naturale, in quanto:

- esiste un nodo centrale che aggrega, persiste e distribuisce le informazioni;
- la board embedded rappresenta un client che comunica verso il server;
- il controllo remoto dei device può essere realizzato attraverso un meccanismo di *polling* dei comandi.

Scelta di un'Architettura a strati per il backend Per quanto riguarda il nodo server, si è scelto di implementare la logica applicativa, di comunicazione e persistenza direttamente su di esso. Per organizzare questi componenti risulta adatta l'implementazione di un'architettura software basata sugli strati (**Layered Architecture**), composta da **controller**, **service**, **repository** e **DTO** ("forma" dei dati).

Tale scelta permette la spartizione delle responsabilità dei vari componenti:

- il **Controller layer** gestisce la comunicazione HTTP e l'interazione coi client;
- il **Service layer** si occupa di implementare la logica applicativa e i vari casi d'uso;
- il **Repository layer** incapsula l'accesso ai dati e la persistenza su database SQLite;
- il **DTO/Model layer** rappresenta i dati scambiati tra i diversi livelli dell'applicazione.

Questa suddivisione consente di separare chiaramente le responsabilità dei vari componenti software, riducendo l'accoppiamento tra i moduli e migliorando manutenibilità, riusabilità ed estendibilità del sistema. Questa architettura è coerente con la scelta di adottare Spring Boot come framework di sviluppo.

Relazione col paradigma MVC Per la parte web, la soluzione adottata potrebbe essere compatibile col paradigma **Model-View-Controller**, in quanto:

- i **controller** svolgono il ruolo di gestori tra la parte di interazione client, la gestione dei dati e la persistenza, fittando perfettamente col ruolo di **Controller**;
- i **DTO** e i dati restituiti dal backend potrebbero rappresentare il **Model**;
- mentre la **dashboard** costituirebbe il livello di **View**.

Tuttavia il server nel suo complesso non è stato progettato come una semplice applicazione **MVC** ma come un backend REST IoT strutturato internamente a strati (layer); pertanto il paradigma dominante resta il **Layered Architecture**, mentre il **MVC** rappresenta soprattutto il modello di organizzazione della parte web (presentation layer).

2.3 Deployment/Topology Diagram

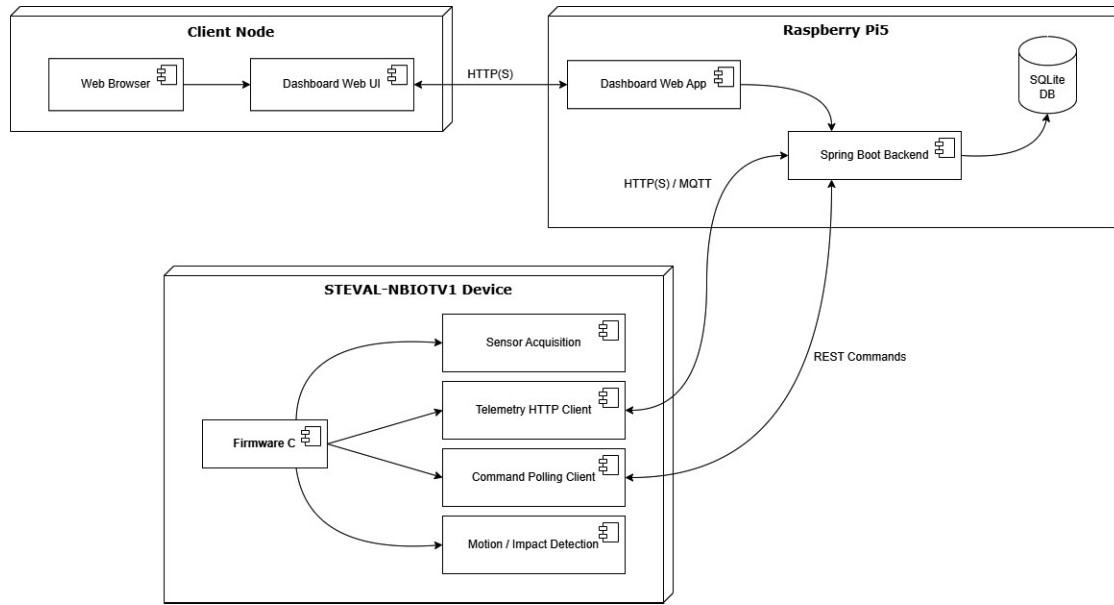


Figure 3: Deployment/Topology Diagram v2

In questa seconda versione del **Deployment/Topology Diagram** (figura 3), i nodi fisici principali sono tre, esattamente come abbiamo già visto nella precedente iterazione:

1. **STEVAL-NBIOTV1 Device** → rappresenta il **nodo embedded** (sviluppato in linguaggio C), ossia il dispositivo in grado di acquisire dati relativi a grandezze fisiche esterne e di trasmetterli verso un nodo server principale.

In questa iterazione aggiornata, sono stati modificati i componenti all'interno di questo nodo fisico; nello specifico, sono presenti: i moduli di acquisizione dei sensori, *Sensor Acquisition*, la trasmissione della telemetria via HTTP(S), *Telemetry HTTP Client*, il recupero dei comandi remoti tramite *polling*, *Command Polling Client*, l'identificazione dei movimenti della board nello spazio, *Motion/Impact detection*, ed infine contiene tutta la logica di raccolta dei parametri ambientali e di stato.

Il ruolo della board sarà quello di acquisire periodicamente parametri quali temperatura, umidità e pressione, oltre ad eventuali informazioni di stato del dispositivo, come stato della batteria e orientamento nello spazio.

2. **Raspberry Pi5** → rappresenta il **nodo server** che ospita il backend applicativo. Si è deciso di sviluppare questa parte in linguaggio **Java** mediante **Spring Boot**, dal momento che era particolarmente coerente con la logica dello stile architetturale **client-server**.

Su questo nodo risiedono, come illustrato in figura 3:

- la **logica applicativa**, implementata e rappresentata dal blocco *Spring Boot Backend*, il cui compito è la gestione del sistema, riceve ed elabora le richieste e contiene le regole di funzionamento (implementata nel **service**);
- il **modulo REST API**, ovvero la parte che rappresenta l'interfaccia di comunicazione esposta verso l'esterno. Gli endpoint comunicano mediante richieste HTTP GET/POST/PUT/... (implementata nel **controller** - *@RestController*);
- la **dashboard web**, identificata nel diagramma dal blocco *Dashboard Web App*, la quale permette la visualizzazione dei dati tramite interfaccia grafica;
- il **database** relazionale *SQLite DB*, utilizzato per la persistenza dei dati.

3. **Client Node** → rappresenta il layer di presentazione, costituito da una dashboard accessibile tramite browser web attraverso la quale è possibile comunicare col backend via HTTP.

Il **Deploymet/Topology Diagram** evidenzia quindi una chiara separazione tra nodo di acquisizione dati, nodo di elaborazione e persistenza, e nodo di fruizione dei servizi. Tale separazione rispecchia una tipica architettura IoT edge-centric, in cui il raspberry svolge un ruolo di nodo centrale locale.

2.4 Component Diagram

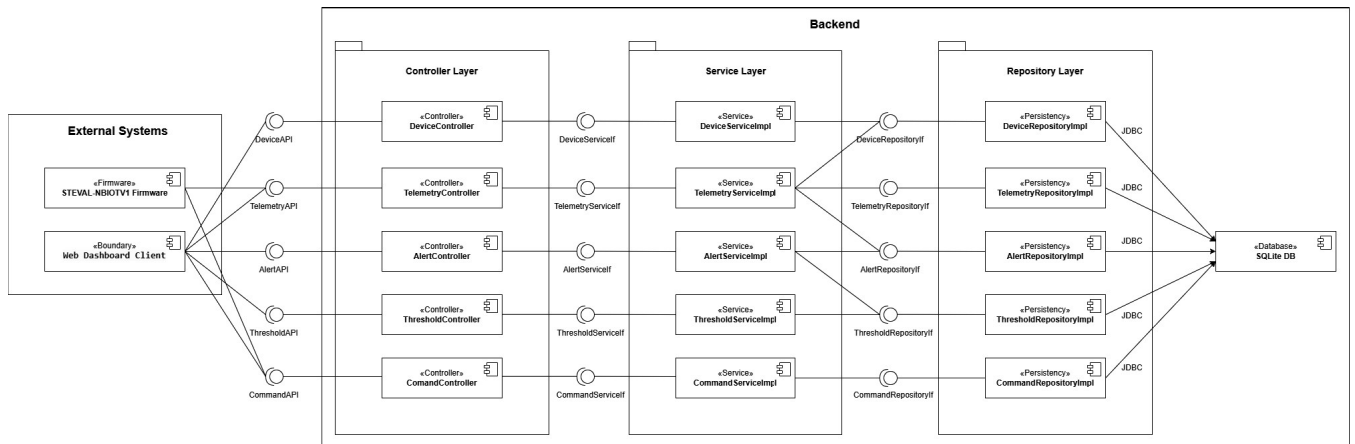


Figure 4: Component Diagram

Il **Component Diagram** rappresenta la struttura logica del server backend ospitato su **Raspberry Pi5** e mostra i principali componenti software, le loro responsabilità e le interfacce di comunicazione tra essi.

Il backend è stato organizzato secondo una **Layered Architecture** che distingue i diversi livelli funzionali dell'applicazione. In particolare i **layer** sono 3:

- **controller layer** → costituisce il punto d'ingresso dell'applicazione lato server dove i **controller** espongono le **API REST**, le quali consentono l'interazione col sistema.

Nello specifico sono presenti le seguenti classi:

- *TelemetryController* → gestisce la ricezione delle telemetrie e la consultazione dei dati raccolti;
- *DeviceController* → consente la consultazione dello stato dei dispositivi;
- *AlertController* → consente l'accesso agli allarmi;
- *ThresholdController* → permette la configurazione delle soglie;
- *CommandController* → gestisce i comandi remoti.

Oss.: i **controller NON** contengono la logica applicativa, ma delegano le operazioni ai **service** adeguati attraverso specifiche interfacce.

- **service layer** → implementa la logica applicativa del sistema introducendo le specifiche interfacce e relativa implementazione (es.: interfaccia *TelemetryService* e relativa classe di implementazione *TelemetryServiceImpl*).

Questo **layer** si occupa di orchestrare i casi d'uso principali quali il salvataggio delle telemetrie, l'aggiornamento dello stato del dispositivo, la generazione degli allarmi, la gestione delle soglie e dei comandi remoti. La scelta di introdurre le interfacce nel **service layer** consente di migliorare la modularità, manutenibilità e possibilità di estensione.

- **repository layer** → si occupa di incapsulare la logica di accesso alla **base di dati**. Sono state introdotte anche in questo caso delle interfacce (es.: interfaccia *TelemetryRepository* e relativa implementazione *TelemetryRepositoryImpl*).

I **repository** isolano completamente il backend dai dettagli della persistenza, nascondendo l'uso di **JDBC** e delle query **SQL**. In questo modo il **service layer** non dipende direttamente dal database ma da un'astrazione.

Vantaggio architetturale: il **Component Diagram** mette in evidenza una struttura a componenti ben separati, con responsabilità specifiche e interfacce esplicite. Tale organizzazione permette la riduzione dell'accoppiamento, migliora la leggibilità del codice, facilita il testing e consente evoluzioni future dell'architettura, come una possibile sostituzione del **DB** introducendo un diverso canale di comunicazione verso l'esterno del backend server.

2.5 Domain Class Diagram

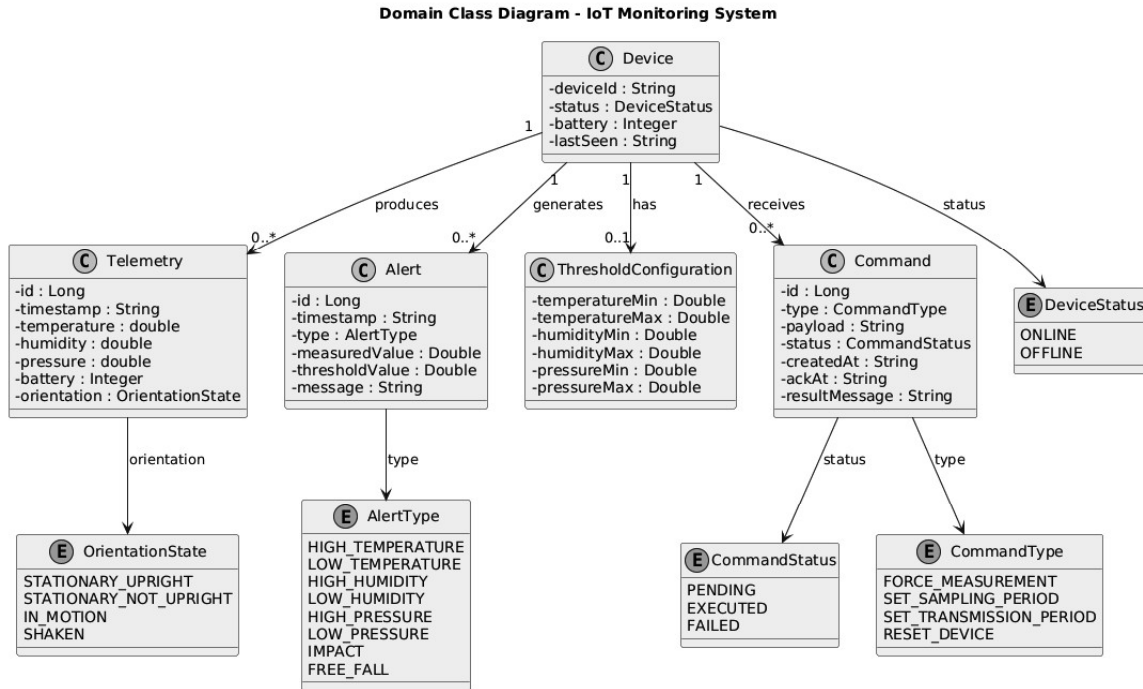


Figure 5: Domain Class Diagram

Il **Domain Class Diagram**, illustrato in figura 5, rappresenta i principali concetti applicativi del sistema di monitoring IoT, indipendentemente dai dettagli implementativi del framework.

Si è quindi scelto di rappresentare le sole entità fondamentali del sistema, quali: **Device**, **Telemetry**, **Alert**, **ThresholdConfiguration** e **Command**, descrivendo per ognuna di esse i rispettivi attributi e relazioni di associazione e molteplicità.

Oltre alle principali classi sono rappresentati anche i tipi enumerativi, quali **DeviceStatus**, **AlertType**, **CommandStatus**, **CommandType** e **OrientationState**, per esplicitare stati e categorie del dominio.

Questo diagramma è quindi utile per descrivere il modello concettuale del sistema e le relazioni tra i suoi oggetti principali.

2.6 Interface Diagram

Il **Diagramma delle Interfacce** descrive invece l'organizzazione architetturale del backend software. Esso mostra la separazione tra i 3 layer implementati: layer di **controllo**, di **servizio** e di **persistenza** in grado di comunicare attraverso le interfacce esposte dai layer di **service** e di **repository**.

Le interfacce del **service** layer, illustrate in figura 6, definiscono le operazioni applicative rese disponibili ai controller **REST (Representational State Transfer API)**, ovvero la gestione della telemetria, dei dispositivi, degli allarmi, delle soglie e dei comandi remoti.

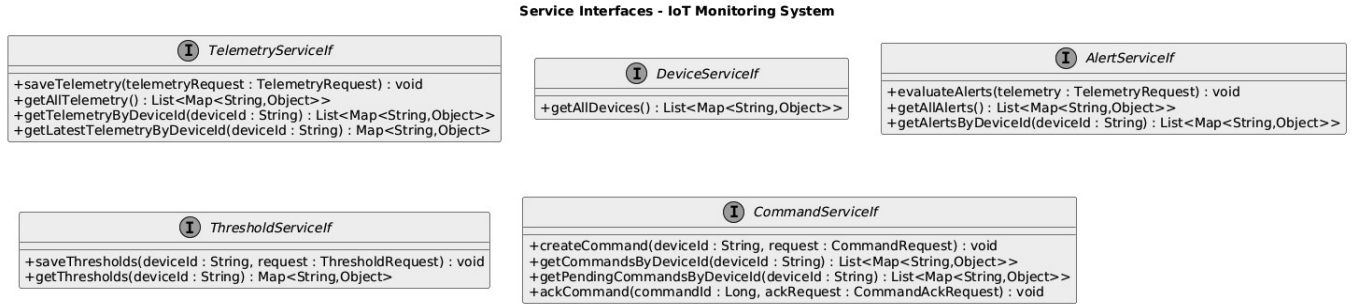


Figure 6: Interface Diagram - Service Layer

Le interfacce del **repository** layer, presenti in figura 7, definiscono invece le operazioni di accesso ai dati, isolando in questo modo tutti i dettagli implementativi del database e delle query *SQL*.

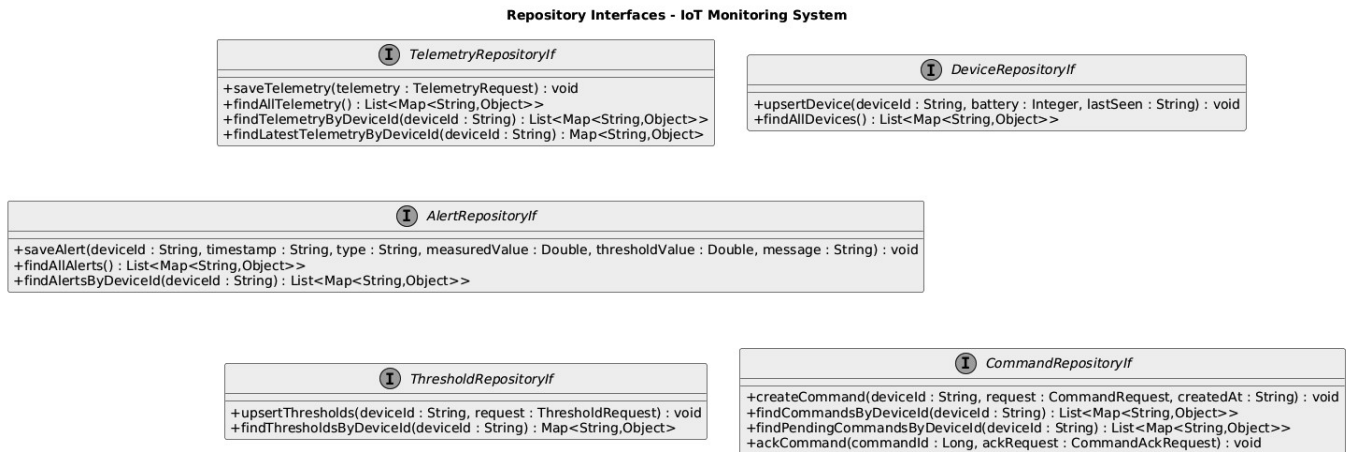


Figure 7: Interface Diagram - Repository Layer

I **controller** dipendono dalle interfacce dei **service**, mentre i **service** dipendono dalle interfacce esposte dai **repository**. Tale scelta progettuale migliora la manutenibilità, estensibilità e testabilità del sistema, coerente con la scelta architetturale implementata (**Layered Architecture**).

Oss.: Non si è invece ritenuto necessario introdurre le interfacce per i **controller** e i **DTO**, in quanto i primi costituiscono adattatori strettamente legati al framework web, mentre i secondi sono semplici oggetti di trasferimento dati privi di logica comportamentale.